



EDUNEXUS - CENTRALIZED ACADEMIC MANAGEMENT & MENTORSHIP PLATFORM



A PROJECT REPORT

Submitted by

MOHAMMED HALITH I	811722104089
REHAAN K	811722104122
ROHINTH R	811722104125
SACHIN B	811722104126

*in partial fulfillment of the requirements for the award degree of
Bachelor of Engineering*

20CS8504 DESIGN PROJECT - 4

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)
SAMAYAPURAM – 621112**

APRIL 2026



EDUNEXUS - CENTRALIZED ACADEMIC MANAGEMENT & MENTORSHIP PLATFORM



A PROJECT REPORT

Submitted by

MOHAMMED HALITH I	811722104089
REHAAN K	811722104122
ROHINTH R	811722104125
SACHIN B	811722104126

*in partial fulfillment of the requirements for the award degree of
Bachelor of Engineering*

20CS8504 DESIGN PROJECT - 4

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)
SAMAYAPURAM – 621112**

APRIL 2026

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)
SAMAYAPURAM - 621112

BONAFIDE CERTIFICATE

The work embodied in the present project report entitled “**EDUNEXUS CENTRALIZED ACADEMIC MANAGEMENT & MENTORSHIP PLATFORM**” has been carried out by the students **MOHAMMED HALITH I, REHAAN K, ROHINTH R, SACHIN B** the work reported here in is original and we declare that the project is their own work, except where specifically acknowledged, and has not been copied from other sources or been previously submitted for assessment.

Date of Viva Voce:

Mrs. P. KARCKEY M.E.,
SUPERVISOR
Assistant Professor
Department Of CSE
K. Ramakrishnan College Of Technology
Samayapuram - 621112

Mr. R. RAJAVARMAN, M.E.,(Ph.D),
HEAD OF THE DEPARTMENT
Assistant Professor (Sr. Grade)
Department Of CSE
K. Ramakrishnan College Of Technology
Samayapuram - 621112

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

The Edunexus Platform is a full-stack web-based platform designed to bridge the gap between mentors and mentees by enabling seamless communication, guidance, and knowledge sharing. In today's fast-paced digital environment, students and professionals often struggle to find the right mentors for career development and skill enhancement. This project addresses that challenge by providing a centralized and user-friendly platform where users can connect based on their interests, expertise, and goals. The system is developed using modern web technologies, with a frontend interface that ensures an intuitive user experience and a backend architecture that efficiently manages data and user interactions. The platform supports features such as user authentication, mentor profile management, mentee requests, and secure communication between users. By integrating these functionalities, the application simplifies the process of finding suitable mentors and facilitates meaningful interactions. The system aims to enhance learning opportunities, promote professional growth, and build a collaborative ecosystem for knowledge exchange..

Keywords: Mentorship Platform, Web Application, Full-Stack Development, User Authentication, Mentor-Mentee Interaction, Career Guidance, Knowledge Sharing, Networking System, Profile Management, Scalable Architecture

ACKNOWLEDGEMENT

We thank our **Dr. N. VASUDEVAN**, Principal, for his valuable suggestions and support during the course of my research work.

We thank our **Mr. R. RAJAVARMAN**, Head of the Department, Assistant Professor (Sr. Grade), Computer Science and Engineering, for his valuable suggestions and support during the course of my research work.

We wish to record my deep sense of gratitude and profound thanks to my Guide **Mrs. P. KARCKEY**, Assistant Professor, Computer Science and Engineering, for her keen interest, inspiring guidance, constant encouragement with my work during all stages, to bring this thesis into fruition.

We are extremely indebted to our project coordinator **Mrs. M. MADHUMATHI**, Assistant Professor, Computer Science and Engineering, for her valuable suggestions and support during the course of my research work.

We also thank the faculty and non-teaching staff members of the Department of Computer Science and Engineering, K Ramakrishnan College of Technology, Samayapuram, for their valuable support throughout the course of my research work.

Finally, we thank our parents, friends and our well wishes for their kind support.

SIGNATURE

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE No.
	ABSTRACT	V
	LIST OF TABLES	viii
	LIST OF FIGURES	Ix
	LIST OF ABBREVIATIONS	X
1	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 OBJECTIVE	2
	1.3 OVERVIEW	2
	1.4 PROBLEM STATEMENT	3
	1.5 IMPLICATION	3
2	LITERATURE SURVEY	4
3	EXISTING SYSTEM	14
	3.1 EXISTING SYSTEM	14
	3.2 DRAWBACKS	16
4	PROBLEMS IDENTIFIED	17
5	PROPOSED SYSTEM	19
	5.1 PROPOSED SYSTEM	19
	5.2 WORK FLOW DIAGRAM	20
6	SYSTEM REQUIREMENTS	22
	6.1 HARDWARE REQUIREMENTS	22
	6.2 SOFTWARE REQUIREMENTS	22
7	SYSTEM IMPLEMENTATIONS	23
	7.1 LIST OF MODULES	23

	7.1.1 User Authentication Module	24
	7.1.2 Profile Management Module	25
	7.1.3 Communication Module	25
	7.1.4 Database Management Module	26
	7.1.5 Admin Module	26
8	SYSTEM TESTING	27
	8.1 UNIT TESTING	27
	8.2 INTEGRATION TESTING	27
	8.3 SYSTEM TESTING	28
	8.4 PERFORMANCE TESTING	28
	8.5 SECURITY TESTING	29
	8.6 USABILITY TESTING	29
9	RESULTS AND DISCUSSION	30
	9.1 RESULT	30
	9.2 DISCUSSION	31
10	CONCLUSION AND FUTURE WORK	32
	10.1 CONCLUSION	32
	10.2 FUTURE ENHANCEMENTS	33
	APPENDIX A - SOURCE CODE	34
	APPENDIX B - SCREENSHOTS	42
	REFERENCES	46

LIST OF FIGURES

FIGURE No.	FIGURE NAME	PAGE No.
3.1	Existing Architecture	14
5.1	Architecture Diagram	20
5.2	Workflow Diagram	21
B.1	Login Page	42
B.2	HOD dashboard	42
B.3	Time Table generation	43
B.4	Student overview page	43
B.5	Mentor dashboard	44
B.6	Chat box	44
B.7	Period-wise attendance	45
B.8	Student dashboard	45

LIST OF ABBREVIATIONS

ABBREVIATION		FULL FORM
SPA	-	Single Page Application
REST API	-	Representational State Transfer Application Programming Interface
JWT	-	JSON Web Token
RBAC	-	Role-Based Access Control
CRUD	-	Create, Read, Update, Delete
DBMS	-	Database Management System
ODM/ORM	-	Object Data/Relational Mapping
LMS	-	Learning management system
HTTP	-	Hyper Text Transfer Protocol
WS	-	WebSocket Protocol
MVC	-	Model View Controller
UI	-	User interface

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

In today's rapidly evolving digital world, access to proper guidance and mentorship plays a crucial role in shaping an individual's academic and professional growth. However, many students and early-career professionals face significant challenges in finding experienced mentors who can provide personalized advice and direction. Traditional mentorship methods are often limited by geographical constraints, lack of structured platforms, and inefficient communication channels.

With the advancement of web technologies and the increasing demand for online learning and networking, digital platforms have emerged as powerful tools to connect individuals across different domains. Despite the availability of various social and professional networking platforms, there is still a lack of dedicated systems focused specifically on structured mentor-mentee relationships.

The project is developed to address this gap by creating a centralized, web-based solution that facilitates meaningful connections between mentors and mentees. By leveraging modern full-stack development technologies, the platform aims to provide a seamless environment for users to interact, share knowledge, and seek guidance based on their interests and expertise.

Furthermore, the proposed system emphasizes personalization and user engagement by incorporating features such as intelligent mentor-mentee matching, structured communication channels, and progress tracking mechanisms. These functionalities ensure that users not only connect with suitable mentors but also maintain consistent interaction and measurable growth over time. By creating a user-friendly interface and integrating secure data management practices, the platform enhances trust and usability. This project is rooted in the need for an accessible, efficient, and scalable mentorship system that overcomes traditional barriers and supports continuous learning and professional development in a digitally connected ecosystem.

1.2 OBJECTIVE

The primary objective of the project is to develop a web-based platform that efficiently connects mentors and mentees, enabling structured and meaningful interactions. The system aims to provide secure user authentication, ensuring safe access and protection of user data. It focuses on simplifying profile creation and management for both mentors and mentees while facilitating seamless communication for effective guidance and knowledge sharing. Additionally, the platform is designed to match users based on their skills, interests, and goals, thereby enhancing the relevance of mentorship. Emphasis is also placed on creating a user-friendly interface and ensuring scalability to support a growing number of users. Ultimately, the project seeks to promote continuous learning, career development, and a collaborative mentorship ecosystem.

1.3 OVERVIEW

The Edunexus Platform is a full-stack web application designed to create an efficient and interactive environment for mentorship. It serves as a digital platform where mentors and mentees can connect, communicate, and collaborate based on shared interests, skills, and career goals. The application focuses on simplifying the mentorship process by providing a centralized system that eliminates traditional barriers such as location constraints and lack of accessibility.

The platform is built using modern web technologies, incorporating a responsive frontend for an enhanced user experience and a robust backend for handling data, user authentication, and system operations. It includes key features such as user registration and login, profile management, mentor-mentee matching, and secure communication. These features work together to ensure a smooth and reliable interaction between users.

The system is designed with scalability and performance in mind, allowing it to support a growing user base without compromising efficiency. By integrating structured data management and intuitive design, the application ensures ease of use for both mentors and mentees.

1.4 PROBLEM STATEMENT

In today's academic and professional environment, individuals often face difficulty in finding the right mentors who can provide proper guidance, career direction, and skill development support. Existing mentorship methods are largely informal, unstructured, and limited by geographical barriers, making it challenging for mentees to connect with experienced professionals. Additionally, there is a lack of dedicated platforms that efficiently match mentors and mentees based on their interests, expertise, and goals.

As a result, many users rely on scattered resources and generalized advice, which may not effectively address their specific needs. This creates a gap in personalized learning and career development opportunities. Therefore, there is a need for a centralized, secure, and user-friendly web-based system that facilitates structured mentor-mentee interaction, enables efficient communication, and ensures meaningful knowledge exchange.

1.5 IMPLICATION

The Edunexus Platform has significant implications in improving the way mentorship is accessed and delivered in the digital era. By providing a centralized platform, it eliminates geographical barriers and makes mentorship opportunities accessible to a wider audience. This leads to increased inclusivity, allowing students and professionals from diverse backgrounds to connect with experienced mentors. The project enhances the quality of guidance by enabling structured and personalized mentor-mentee interactions, which can result in better career decisions, skill development, and overall professional growth. It also promotes a culture of knowledge sharing and collaboration, strengthening the learning ecosystem.

CHAPTER 2

LITERATURE SURVEY

2.1 SCALABILITY AND LOAD HANDLING IN WEB APPLICATIONS

Reddy, Kumar et al. (2021) discuss the importance of scalability and load handling in modern web applications, especially in platforms that experience dynamic user growth such as mentorship systems. The authors explain that scalability refers to the system's ability to handle an increasing number of users, requests, and data without affecting performance. As applications like MentorConnect grow, they must efficiently manage higher loads while maintaining speed, reliability, and responsiveness.

The study highlights two primary types of scalability: vertical scaling and horizontal scaling. Vertical scaling involves upgrading system resources such as CPU and memory, while horizontal scaling involves adding more servers to distribute the load. The research emphasizes that horizontal scaling is more effective for large-scale applications, as it provides better fault tolerance and flexibility. Load balancing techniques are also discussed, where incoming requests are distributed across multiple servers to prevent overloading and ensure smooth operation.

Furthermore, the paper explains the role of distributed system architecture in achieving scalability. Microservices architecture is highlighted as a modern approach, where applications are divided into smaller, independent services that can be scaled individually. This approach improves system maintainability and allows efficient resource utilization. The use of containerization technologies like Docker and orchestration tools such as Kubernetes is also mentioned for managing scalable deployments.

Additionally, the study focuses on performance optimization strategies such as caching, database replication, and asynchronous processing. Caching reduces the need for repeated computations, while replication ensures data availability across multiple servers. Asynchronous processing allows the system to handle multiple tasks concurrently, improving responsiveness.

2.2 API DESIGN AND INTEGRATION IN WEB APPLICATIONS

Gupta, Verma et al. (2022) discuss the crucial role of Application Programming Interfaces (APIs) in modern web application development. The authors explain that APIs act as a bridge between different components of a system, enabling seamless communication between the frontend, backend, and external services. In full-stack applications like MentorConnect, APIs are essential for handling user requests, transferring data, and ensuring smooth interaction between system modules.

The study highlights the use of RESTful APIs, which follow standard HTTP methods such as GET, POST, PUT, and DELETE to perform operations on data. REST architecture ensures simplicity, scalability, and flexibility, making it widely adopted in web development. The research also discusses the importance of designing clean and well-structured API endpoints, which improves maintainability and ease of integration. Proper API documentation is emphasized as it helps developers understand and use the system effectively.

Furthermore, the paper explains how APIs enable integration with third-party services such as authentication providers, payment gateways, and notification systems. This enhances the functionality of web applications without requiring the development of complex features from scratch. In mentorship platforms, APIs can be used for sending notifications, managing user sessions, and integrating communication tools.

Additionally, the study focuses on API security measures such as authentication tokens, rate limiting, and data validation. These practices help prevent unauthorized access and protect the system from malicious attacks. The use of API gateways and middleware is also discussed for managing requests, monitoring performance, and ensuring secure data flow. In addition to these aspects, the study also emphasizes the role of APIs in enhancing system performance and scalability in large-scale applications. By enabling modular architecture, making it easier to update or expand specific features without affecting the entire application. This is particularly beneficial for platforms like MentorConnect, where user demand and data volume may grow over time. The use of caching mechanisms and optimized API calls further improves response time and overall system efficiency.

2.3 PERFORMANCE OPTIMIZATION IN WEB APPLICATIONS

Sharma, Patel et al. (2021) discuss the importance of performance optimization in modern web applications, particularly in systems that handle multiple users and real-time interactions such as mentorship platforms. The authors explain that application performance directly impacts user experience, engagement, and retention. Slow loading times, delays in data processing, and inefficient system behavior can negatively affect user satisfaction and lead to reduced platform usage.

The study highlights various techniques used to improve the performance of web applications. One of the key approaches is frontend optimization, which includes minimizing HTTP requests, compressing files, and using efficient rendering techniques. Technologies such as lazy loading and code splitting are employed to ensure that only necessary resources are loaded, thereby reducing page load time. The use of Content Delivery Networks (CDNs) is also emphasized to distribute content efficiently across different geographical locations.

Furthermore, the research discusses backend optimization strategies such as efficient server-side processing, database query optimization, and caching mechanisms. Caching techniques, including in-memory caching and browser caching, help reduce repeated data processing and improve response time. Load balancing is another important technique that distributes user requests across multiple servers to prevent system overload and ensure high availability.

The paper also emphasizes the role of asynchronous processing and event-driven architectures in improving system responsiveness. By handling tasks such as data fetching and processing asynchronously, applications can provide a smoother user experience. The study highlights the importance of monitoring tools and performance testing to identify bottlenecks and optimize system components effectively.

2.4 FULL-STACK WEB APPLICATION DEVELOPMENT

Anderson and Miller (2020) explore the importance of full-stack web application development in building modern, scalable, and interactive digital platforms. The authors explain that full-stack development involves working with both the frontend (client-side) and backend (server-side) components of an application, enabling developers to create complete and fully functional systems. This approach ensures seamless integration between user interfaces, server logic, and database management.

The study highlights that frontend technologies such as HTML, CSS, and JavaScript frameworks like React are used to design responsive and user-friendly interfaces. These technologies focus on enhancing user experience by providing dynamic content, smooth navigation, and interactive elements. On the other hand, backend technologies such as Node.js and Express.js are responsible for handling business logic, processing user requests, and managing communication with the database.

Furthermore, the research emphasizes the role of RESTful APIs in connecting the frontend and backend components. APIs act as intermediaries that allow data exchange between the client and server in a structured and efficient manner. This modular architecture improves maintainability and allows developers to update individual components without affecting the entire system. The study also discusses the integration of databases such as MongoDB or PostgreSQL for storing and managing application data.

Additionally, the paper highlights key aspects such as scalability, performance optimization, and security in full-stack development. Techniques like caching, load balancing, and asynchronous processing are used to improve application performance. The use of version control systems and development tools ensures efficient collaboration and code management.

2.5 USER AUTHENTICATION AND SECURITY

Brown and Davis (2020) analyze the critical role of user authentication and security mechanisms in modern web applications, especially in platforms that handle sensitive user data such as mentorship systems. The authors explain that authentication is the process of verifying the identity of users before granting access to system resources. A secure authentication system ensures that only authorized users can access the platform, thereby protecting personal information and maintaining system integrity.

The study highlights commonly used authentication techniques such as password-based login systems, multi-factor authentication (MFA), and token-based authentication methods like JSON Web Tokens (JWT). Password hashing algorithms, including bcrypt and SHA-based methods, are used to securely store user credentials, preventing exposure in case of data breaches. Token-based authentication enables stateless communication between the client and server, improving scalability and performance.

Furthermore, the research emphasizes the importance of authorization mechanisms, particularly role-based access control (RBAC), which restricts system access based on user roles such as mentor and mentee. This ensures that users can only access features and data relevant to their role. The study also discusses session management techniques to maintain secure user sessions and prevent unauthorized reuse of session data.

In addition, the paper addresses key security practices such as data encryption, secure communication protocols (HTTPS), and protection against common web vulnerabilities including SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF). Input validation and sanitization are highlighted as essential methods to prevent malicious data from compromising the system.

2.6 RECOMMENDATION AND MATCHING SYSTEMS

Lee, Kim et al. (2022) examine the importance of recommendation and matching systems in modern web applications, particularly in platforms that require personalized user interactions such as mentorship systems. The authors explain that recommendation systems are designed to analyze user data, preferences, interests, and behavioral patterns to provide relevant suggestions. In mentorship platforms like Edunexus, these systems play a crucial role in connecting mentees with suitable mentors based on compatibility factors such as skills, expertise, goals, and experience levels.

The study highlights various techniques used in recommendation systems, including collaborative filtering, content-based filtering, and hybrid approaches. Collaborative filtering analyzes the behavior and preferences of similar users to generate recommendations, while content-based filtering focuses on matching user profiles and attributes. Hybrid systems combine both approaches to improve accuracy and overcome individual limitations. These techniques help ensure that users receive meaningful and relevant matches, thereby enhancing the overall effectiveness of the platform.

Furthermore, the research emphasizes the importance of data quality and pre processing in achieving accurate recommendations. Clean, structured, and updated user data is essential for the proper functioning of matching algorithms. The study also discusses the use of machine learning models to continuously improve recommendation accuracy based on user interactions and feedback over time. Additionally, the study highlights the significance of dynamic and adaptive recommendation systems that evolve with user behavior. As users interact with the platform—by updating profiles, engaging in sessions, or providing feedback—the system can refine its matching results to deliver more precise and personalized recommendations. This continuous learning approach not only improves the accuracy of mentor-mentee pairing but also enhances user satisfaction and engagement.

2.7 DATABASE MANAGEMENT SYSTEMS

Clark and Evans (2019) discuss the fundamental role of Database Management Systems (DBMS) in supporting modern web applications. The authors explain that a DBMS is responsible for storing, organizing, and managing large volumes of structured and unstructured data efficiently. In web-based platforms like Edunexus, databases handle critical information such as user profiles, mentor-mentee relationships, communication records, and system logs. Efficient data management ensures smooth application performance and reliable user experience.

The study highlights the use of both relational (SQL) and non-relational (NoSQL) databases depending on application requirements. Relational databases such as MySQL and PostgreSQL are known for maintaining data integrity through structured schemas and relationships, while NoSQL databases like MongoDB provide flexibility and scalability for handling dynamic data. The choice of database depends on factors such as data complexity, scalability needs, and performance requirements.

Furthermore, the research emphasizes key database design principles such as normalization, indexing, and efficient query optimization. Normalization helps reduce data redundancy and ensures consistency, while indexing improves data retrieval speed. Query optimization techniques are used to minimize response time and enhance system efficiency, especially when dealing with large datasets.

The paper also discusses the importance of data integrity, consistency, and security. Mechanisms such as constraints, transactions, and ACID (Atomicity, Consistency, Isolation, Durability) properties ensure that the database remains reliable even during system failures.

In addition to these aspects, the study also highlights the role of modern DBMS in supporting scalability and high availability in web applications. As platforms like Edunexus grow, the database must efficiently handle increasing numbers of users and large volumes of data. Techniques such as database replication, sharding, and load balancing are discussed as effective methods to distribute data and maintain system performance under heavy load.

2.8 USER INTERFACE AND USER EXPERIENCE DESIGN

Wilson and Taylor (2021) emphasize the critical role of User Interface (UI) and User Experience (UX) design in the success of modern web applications. The authors explain that UI/UX design focuses on creating visually appealing, intuitive, and user-friendly interfaces that enhance overall user satisfaction. A well-designed interface ensures that users can easily navigate through the system, understand its functionalities, and accomplish their tasks efficiently without confusion.

The study highlights that responsive design is a key aspect of UI/UX, allowing applications to function seamlessly across various devices such as desktops, tablets, and smartphones. By using flexible layouts, adaptive components, and optimized graphics, developers can ensure consistent performance and appearance regardless of screen size. This is particularly important for web-based platforms like Edunexus, where users may access the system from different devices.

Furthermore, the research discusses essential design principles such as simplicity, consistency, accessibility, and visual hierarchy. Simplicity ensures that the interface is not cluttered, making it easier for users to focus on important features. Consistency in design elements such as colors, fonts, and layouts improves usability and reduces the learning curve. Accessibility ensures that the platform is usable by individuals with different abilities, including those with visual or physical impairments.

The paper also emphasizes the importance of user-centered design, where the system is developed based on user needs and feedback. Techniques such as wireframing, prototyping, and usability testing are used to evaluate and improve the interface during development. Feedback mechanisms, including notifications, alerts, and error messages, play a vital role in guiding users and improving their interaction with the system.

2.9 REAL-TIME COMMUNICATION SYSTEMS

Garcia and Thomas (2022) explore the growing importance of real-time communication systems in modern web applications, particularly in platforms that require continuous user interaction such as mentorship systems. The authors explain that real-time communication enables instant data exchange between users without requiring page refreshes, thereby significantly improving responsiveness and user engagement. Technologies such as WebSockets, Socket.IO, and real-time messaging protocols are widely used to establish persistent connections between the client and server, allowing seamless bidirectional communication.

The study highlights that real-time features such as instant messaging, live notifications, typing indicators, and activity status updates play a crucial role in enhancing user experience. In mentorship platforms, these features allow mentees to receive immediate guidance and feedback from mentors, making the interaction more effective and dynamic. The ability to conduct live discussions and resolve queries instantly improves learning efficiency and strengthens the mentor-mentee relationship.

Furthermore, the research discusses the architectural aspects of implementing real-time systems. It emphasizes the use of event-driven programming and asynchronous communication models to handle multiple user requests simultaneously. Scalable server infrastructures, such as cloud-based servers and microservices architecture, are essential for supporting a large number of concurrent users. The study also explains the integration of push notification services to keep users informed about new messages, requests, and updates even when they are not actively using the platform.

In addition to these features, the study also emphasizes the importance of reliability and fault tolerance in real-time communication systems. Since mentorship platforms depend heavily on continuous interaction, it is essential to ensure minimal latency and uninterrupted connectivity. Techniques such as message queuing, load balancing, and automatic reconnection mechanisms help maintain stable communication even during network fluctuations or server issues.

2.10 CLOUD COMPUTING IN WEB APPLICATIONS

Kumar, Singh et al. (2021) discuss the significant role of cloud computing in the development and deployment of modern web applications. The authors explain that cloud platforms provide scalable infrastructure, enabling applications to handle increasing user demands without compromising performance. By utilizing cloud services, developers can deploy applications efficiently while reducing the need for physical hardware and maintenance costs. This approach enhances system flexibility and ensures high availability.

The study highlights that cloud computing supports features such as data storage, backup, and real-time processing. Services like Infrastructure as a Service (IaaS) and Platform as a Service (PaaS) allow developers to build and manage applications with ease. The integration of cloud databases ensures secure and efficient data management, which is crucial for applications handling large volumes of user information, such as mentorship platforms.

Furthermore, the research emphasizes the importance of load balancing and distributed computing in maintaining system performance during high traffic conditions. It also discusses the role of cloud security measures, including data encryption and access control, in protecting sensitive user data. Challenges such as service outages, data privacy concerns, and dependency on third-party providers are also addressed.

The study concludes that cloud computing is a fundamental technology for modern web applications, providing scalability, reliability, and cost-effectiveness. It plays a vital role in supporting platforms like MentorConnect by ensuring seamless operation, efficient data handling, and improved user experience. In addition to these advantages, the study also highlights the role of cloud computing in enabling continuous integration and continuous deployment (CI/CD) practices. Cloud platforms provide automated tools that allow developers to test, update, and deploy applications quickly and efficiently. This ensures that new features, bug fixes, and improvements can be delivered to users without significant downtime, enhancing the overall reliability of the system.

CHAPTER 3

EXISTING SYSTEM

3.1 EXISTING SYSTEM

The existing system for mentorship and academic interaction is primarily based on a basic web portal that provides limited functionality such as login access, static pages, and basic information retrieval. All users, including students, advisors, and administrators, access the system through a single entry point without any role-based customization. This results in a lack of personalized experience and reduced efficiency in handling different user requirements.

In the current setup, functionalities such as attendance, marks, and timetable management are handled through separate applications, which are not integrated into a unified system. This fragmentation leads to poor coordination between modules and increases the complexity for users who need to switch between multiple platforms. The system also suffers from the absence of a proper backend integration, resulting in limited data processing capabilities and inefficient data management.

Additionally, the existing system lacks effective communication and interaction features between users. There is no provision for real-time messaging, notifications, or direct mentor-mentee engagement, which limits the overall effectiveness of the mentorship process. Users often rely on external platforms for communication, leading to a disconnected and less streamlined experience.

Moreover, the system does not support advanced features such as intelligent matching, progress tracking, or personalized recommendations. This makes it difficult for students to find suitable mentors based on their interests and goals. The absence of analytics and reporting tools further restricts the ability of administrators to monitor user activity and system performance. Overall, the current system is limited in scalability, flexibility, and user engagement, highlighting the need for a more integrated and feature-rich solution like Edunexus.

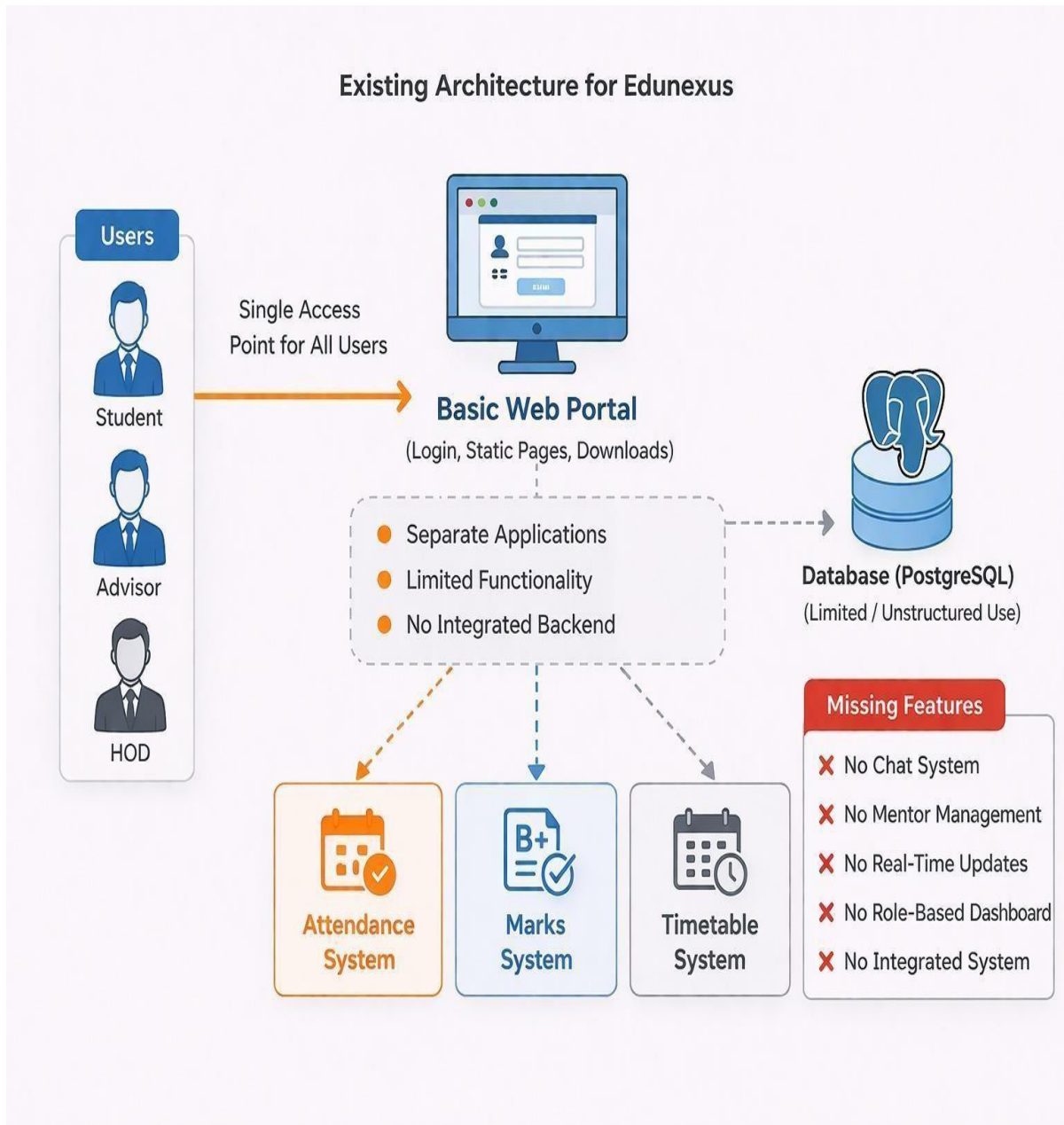


Figure 3.1 Existing System Architecture

3.2 DRAWBACKS:

- Lack of a centralized and integrated mentorship platform
- No real-time communication or chat system
- Absence of mentor-mentee matching functionality
- Limited features with only basic web portal access
- No role-based dashboard for different users

CHAPTER 4

PROBLEMS IDENTIFIED

Existing academic systems face several challenges that reduce their effectiveness and usability. One major issue is the lack of a centralized platform, making it difficult for users to find suitable mentors. There is also an absence of structured communication mechanisms, leading to ineffective interactions between mentors and mentees. Many systems do not provide personalized matching, resulting in irrelevant connections. Security concerns arise due to weak authentication and data protection measures. Additionally, poor user interface and user experience design reduce user engagement. The lack of real-time communication features further limits interaction quality. Scalability issues affect system performance when handling many users. Inefficient data management can slow down operations, and the absence of feedback or progress tracking reduces mentorship effectiveness. Moreover, limited use of modern technologies restricts the overall capability of existing systems.

The existing system lacks a proper and structured communication mechanism between mentors and mentees. Users often rely on external platforms for interaction, which leads to fragmented and inconsistent communication. There is no real-time messaging or notification system to ensure timely responses. This results in delays in guidance and reduced interaction quality. Important discussions and feedback are not properly recorded or tracked. The absence of integrated communication tools makes collaboration difficult. Overall, ineffective communication reduces the efficiency and impact of the mentorship process.

Users are unable to find suitable mentors based on their skills, interests, and career goals. Instead, they rely on random or manual selection, which often leads to irrelevant or less effective connections. There is no algorithm or system to analyze user profiles and recommend appropriate matches. This reduces the quality of mentorship and user satisfaction. Additionally, it increases the time and effort required to find the right mentor. The lack of personalization limits the overall

efficiency and success of the mentorship process.

Data related to users, communication, and activities is often stored in separate systems without a unified structure. This leads to data redundancy, inconsistency, and difficulty in retrieval. The absence of proper database design affects system performance and reliability. Additionally, different modules are not well integrated, causing poor data flow between them. Users may experience delays and errors due to unorganized data handling. Overall, poor data management and integration reduce the efficiency and effectiveness of the system.

Sensitive information such as personal details and login credentials may not be properly encrypted. There is no robust authentication or authorization mechanism to prevent unauthorized access. This increases the risk of data breaches and misuse of information. Additionally, the system may not protect against common cyber threats like hacking or phishing.

As user traffic increases, the system may experience slow response times and performance issues. It lacks scalable architecture, making it difficult to expand resources when demand rises. This can lead to system crashes or delays during peak usage. Additionally, inefficient data handling and processing further reduce performance. The system may not support real-time operations effectively under heavy load.

The existing system provides a poor user interface and lacks an intuitive design, making it difficult for users to navigate. The layout is not user-friendly, and important features are not easily accessible. This increases the learning curve for new users and reduces overall usability. The system may not be responsive across different devices, affecting accessibility. Additionally, lack of visual clarity and feedback mechanisms leads to confusion during usage. Users may face difficulty in completing tasks efficiently.

CHAPTER 5

PROPOSED SYSTEM

5.1 PROPOSED SYSTEM

The proposed system, Edunexus Platform, is designed as a fully integrated and centralized web-based platform that overcomes the limitations of the existing system. It provides a unified environment where students, mentors, and administrators can access all functionalities through a single system with role-based access control. This ensures a personalized experience for each user.

The system is developed using modern full-stack technologies, with a responsive frontend (HTML, CSS, React/JavaScript) and a robust backend (Node.js/Express or Python-based services). It includes key modules such as user authentication, profile management, mentor-mentee matching, real-time communication, and dashboard reporting. These modules are interconnected to ensure smooth data flow and efficient operation.

The backend server handles API routing, authentication, authorization, and data processing, while the database (MongoDB/PostgreSQL) stores user data, mentorship records, and communication logs in a structured manner. Features like real-time chat, notifications, and task management enhance user interaction and engagement.

Additionally, the system ensures data security through encryption and role-based access, while scalable architecture allows it to handle increasing users efficiently. Overall, the proposed system provides a reliable, efficient, and user-friendly solution for modern mentorship, improving accessibility, communication, and overall system performance.

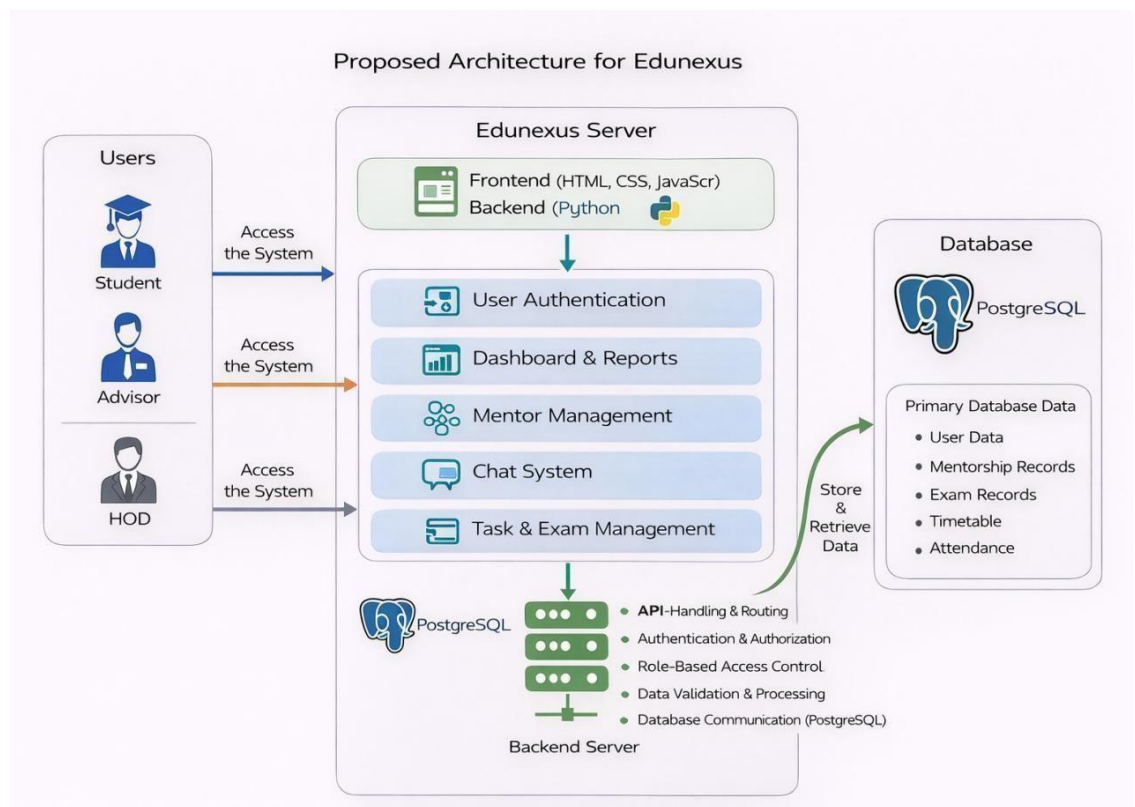


Figure 5.1 Architecture Diagram of Edunexus Platform

5.2 WORK FLOW

The workflow of the Edunexus Platform begins when users (students, mentors, and admins) access the platform. Initially, users perform registration and login, after which the system verifies their identity through the user authentication module. Once authenticated, users are granted access to their respective dashboards based on their roles.

From the dashboard, students can search and match with suitable mentors based on their interests and requirements. After successful matching, both mentors and mentees can engage in real-time communication, enabling effective interaction and knowledge sharing. The system also allows users to manage activities through the task and progress tracking module, where tasks, goals, and progress are monitored.

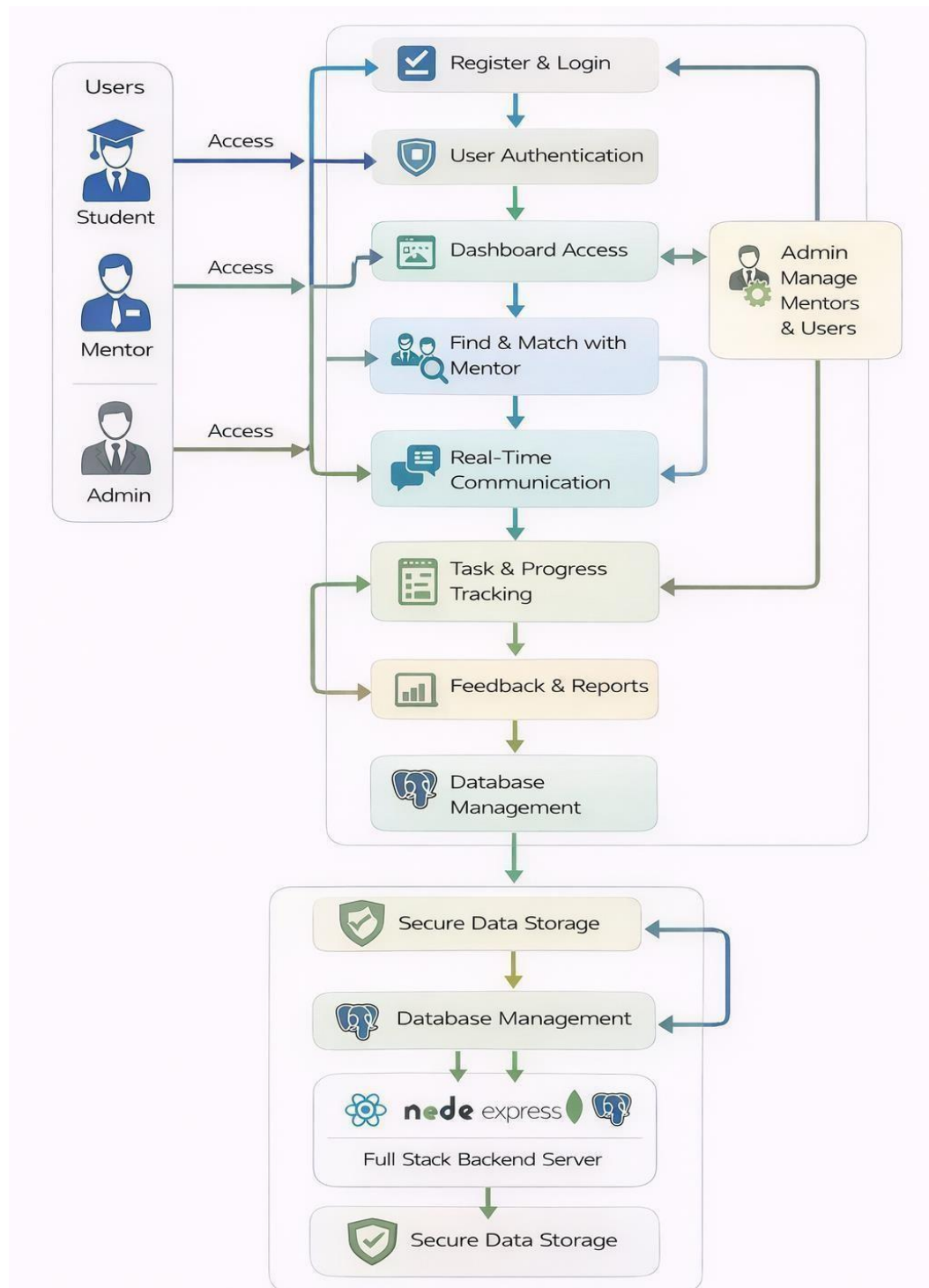


Figure 5.2 Workflow of the Proposed System

CHAPTER 6

SYSTEM REQUIREMENTS

6.1 HARDWARE REQUIREMENTS

1. **Processor:** Intel i5 / Ryzen 5 or above
2. **RAM:** Minimum 8 GB
3. **Hard Disk:** 256 GB (SSD Recommended)
4. **System Type:** 64-bit Architecture
5. **Network:** Stable Internet Connection

6.2 SOFTWARE REQUIREMENTS

1. **Operating System:** Windows / Linux / macOS
2. **Programming Language:** JavaScript
3. **Frontend:** HTML, CSS, React.js
4. **Backend:** Node.js, Express.js
5. **Database:** MongoDB / PostgreSQL
6. **API Tools:** Postman
7. **IDE:** Visual Studio Code
8. **Version Control:** Git & GitHub
9. **Web Browser:** Google Chrome / Edge

CHAPTER 7

SYSTEM IMPLEMENTATIONS

The implementation of the Edunexus Platform involves the development of a full-stack web application using modern technologies to ensure efficient performance, scalability, and user-friendly interaction. The system is divided into two main components: the frontend and the backend, which work together to deliver seamless functionality.

The frontend is developed using HTML, CSS, and React.js, providing a responsive and interactive user interface. It allows users to register, log in, create profiles, search for mentors, and interact with the system easily. The design focuses on simplicity, clarity, and accessibility to enhance user experience.

The backend is implemented using Node.js and Express.js, which handle server-side logic, user authentication, request processing, and communication with the database. RESTful APIs are developed to enable efficient data exchange between the frontend and backend components. Secure authentication mechanisms such as token-based authentication are used to protect user data and ensure authorized access.

The database layer is managed using MongoDB or PostgreSQL, where user information, mentor-mentee relationships, and communication data are stored and retrieved efficiently. Proper database design and indexing techniques are applied to ensure fast data access and system performance.

7.1 LIST OF MODULES

The MentorConnect system is divided into several functional modules to ensure organized and efficient operation. The User Authentication Module manages user registration, login, and secure access to the platform. The Profile Management Module allows mentors and mentees to create and update their

profiles with relevant details. The Matching Module connects users based on their skills, interests, and goals. The Communication Module enables real-time interaction through messaging and notifications. The Database Management Module handles storage and retrieval of user and system data. Additionally, the Admin Module monitors system activities and manages users. These modules work together to provide a seamless and user-friendly mentorship experience.

- User Authentication Module
- Profile Management Module
- Communication Module
- Database Management Module
- Admin Module

7.1.1 USER AUTHENTICATION MODULE

The User Authentication Module is responsible for managing secure access to the Edunexus Platform. It allows users to register by providing essential details such as name, email, and password. The module verifies user credentials during login to ensure only authorized access. Passwords are securely stored using encryption techniques to protect user data. It also supports token-based authentication to maintain user sessions efficiently. The module includes validation mechanisms to prevent invalid or malicious inputs. Additionally, it provides features like password reset and account recovery. Overall, this module ensures data security, user privacy, and reliable access control within the system.

Furthermore, the User Authentication Module plays a vital role in maintaining system integrity by implementing role-based access control, ensuring that different users such as students, mentors, and administrators have appropriate permissions within the platform. This helps in restricting unauthorized actions and protecting sensitive data from misuse. The module also logs authentication activities, which can be used for monitoring, auditing, and detecting suspicious behavior.

7.1.2 PROFILE MANAGEMENT MODULE

The Profile Management Module allows users to create, view, and update their personal and professional information within the Edunexus Platform. It enables mentors to showcase their expertise, skills, experience, and availability, while mentees can specify their interests, goals, and learning preferences. The module ensures that profile data is stored and retrieved efficiently from the database. It includes validation mechanisms to maintain accurate and consistent information. Users can edit their profiles at any time to keep their details up to date. The module also supports profile visibility and customization features. Overall, it plays a key role in enabling effective matching and interaction between mentors and mentees.

7.1.3 COMMUNICATION MODULE

The Communication Module enables seamless interaction between mentors and mentees within the Edunexus Platform. It provides features such as real-time messaging, chat functionality, and notifications to ensure effective communication. Users can send and receive messages instantly, allowing quick clarification of doubts and continuous guidance. The module supports structured conversations to maintain clarity and context. It also includes notification alerts for new messages, requests, and updates. Secure communication protocols are implemented to protect user data and privacy. Additionally, the module ensures reliable message delivery and synchronization. Overall, it enhances user engagement and facilitates meaningful mentor-mentee interactions.

7.1.4 DATABASE MANAGEMENT MODULE

The Database Management Module is responsible for storing, organizing, and managing all data within the Edunexus Platform. It handles user information, profiles, mentor-mentee relationships, messages, and system logs efficiently. The module uses databases such as MongoDB or PostgreSQL to ensure reliable data storage and quick retrieval. It implements proper data structuring, indexing, and query optimization techniques to improve performance. Security measures such as

data encryption and access control are applied to protect sensitive information. The module also supports backup and recovery mechanisms to prevent data loss. Additionally, it ensures data consistency and integrity across the system. Overall, this module plays a crucial role in maintaining smooth and efficient system operations.

7.1.5 ADMINMODULE

The Admin Module is responsible for monitoring and managing the overall functioning of the Edunexus Platform. It allows the administrator to manage user accounts, including approving, updating, or removing users when necessary. The module provides control over mentor and mentee activities to ensure proper usage of the system. It also monitors system performance, user interactions, and data flow. The admin can handle reports, complaints, and feedback submitted by users. Security and access control are maintained by overseeing authentication and system policies. Additionally, the module supports data management and system updates. Overall, it ensures smooth operation, reliability, and proper governance of the platform.

CHAPTER 8

SYSTEM TESTING

System testing is an essential phase in the development of the Edunexus Platform to ensure that all components function correctly and meet the specified requirements. It involves verifying the complete system, including frontend, backend, and database integration, to identify and resolve any errors or inconsistencies. Various testing methods are applied during this phase. Unit testing is performed to test individual modules such as authentication, profile management, and communication features. Integration testing ensures that different modules interact correctly with each other. System testing evaluates the overall functionality of the application as a whole, while user acceptance testing (UAT) is conducted to ensure the system meets user expectations and requirements.

8.1 UNIT TESTING

Unit testing is a software testing technique in which individual components or modules of the Edunexus Platform are tested independently to ensure they function correctly. Each module, such as user authentication, profile management, and communication, is tested separately to verify its logic and expected output. This helps in identifying errors at an early stage of development.

In this process, developers write test cases for specific functions, such as login validation, data input handling, and API responses. Automated testing tools and frameworks may be used to execute these tests efficiently. Unit testing ensures that each function performs as intended under different conditions.

8.2 INTEGRATION TESTING

Integration testing is a software testing process in which different modules of the MentorConnect system are combined and tested together to ensure they work correctly as a unified system. After individual modules such as user authentication, profile management, communication, and database management

are tested separately, integration testing verifies the interaction between these components. This testing ensures that data flows properly between the frontend, backend, and database. For example, it checks whether user login details are correctly validated and stored, or whether messages sent through the communication module are properly saved and retrieved from the database. It helps identify issues such as data mismatch, API errors, and communication failures between modules.

8.3 SYSTEM TESTING

System testing is conducted to evaluate the complete Edunexus Platform application as an integrated system to ensure that it meets all functional and non-functional requirements. In this phase, the entire platform—including frontend, backend, and database—is tested together to verify that all modules operate smoothly and deliver the expected results. The testing process involves validating core functionalities such as user registration and login, profile creation and updates, mentor-mentee matching, and real-time communication. It ensures that data flows correctly between different components and that all features work without errors. Special attention is given to user workflows to confirm that the system provides a seamless and intuitive experience.

8.4 PERFORMANCE TESTING

Performance testing is carried out to evaluate the efficiency, speed, and stability of the Edunexus Platform under various conditions. This testing ensures that the application performs well when accessed by multiple users simultaneously and maintains responsiveness during high traffic. The testing process includes measuring response time, system throughput, and resource utilization such as CPU and memory usage. Key functionalities like user login, profile loading, mentor matching, and messaging are tested to ensure they execute quickly without delays. Load testing is performed to analyze how the system behaves under normal and peak user loads, while stress testing checks the system's limits by increasing the number of users beyond expected capacity.

8.5 SECURITY TESTING

Security testing is conducted to ensure that the Edunexus Platform is protected against unauthorized access, data breaches, and potential cyber threats. This phase focuses on verifying that all security mechanisms implemented in the application function correctly and safeguard sensitive user information such as login credentials, personal details, and communication data. The testing process includes validating authentication and authorization mechanisms to ensure that only registered users can access the system and that role-based access control is properly enforced. It also involves checking for common vulnerabilities such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF). Input validation and data sanitization are tested to prevent malicious data from entering the system.

8.6 USABILITY TESTING

Usability testing is performed to evaluate how easily users can interact with the Edunexus Platform and to ensure a smooth and intuitive user experience. This testing focuses on assessing the design, navigation, and overall functionality of the system from the user's perspective. The process involves observing users as they perform tasks such as registration, login, profile creation, searching for mentors, and communication. It helps identify any difficulties or confusion faced by users while navigating the platform. Feedback is collected to improve interface design, layout, and functionality.

CHAPTER 9

RESULT AND DISCUSSION

9.1 RESULT

The development and testing of the Edunexus Platform demonstrate that the platform successfully fulfills its intended objectives of connecting mentors and mentees through a structured and user-friendly interface. All core functionalities, including user authentication, profile management, mentor-mentee matching, and communication, were implemented and tested effectively. The system showed reliable performance with smooth navigation and quick response times during normal and moderate user loads.

The testing results indicate that the application provides secure access through proper authentication and data protection mechanisms. The matching module was able to generate relevant connections based on user profiles, improving the effectiveness of mentorship. Real-time communication features enabled seamless interaction between users, enhancing engagement and usability. It is evident that the system overcomes many limitations of traditional mentorship methods by offering a centralized and accessible platform. However, some areas such as advanced recommendation algorithms, enhanced UI features, and scalability improvements can be further developed to improve system performance and user experience.

The Edunexus Platform ensures a high level of accuracy in communication between mentors and mentees through its structured and reliable messaging features. The communication module is designed to deliver messages in real-time without data loss or delay, ensuring that information is transmitted correctly. Proper synchronization mechanisms are implemented so that messages are consistently updated across both sender and receiver interfaces. The application is designed using optimized frontend and backend technologies, ensuring smooth interaction between users and the system.

9.2 DISCUSSION

The overall evaluation of the Edunexus Platform indicates that it effectively addresses the challenges associated with traditional mentorship methods by providing a centralized and digital platform. The system successfully integrates essential features such as secure authentication, efficient profile management, accurate mentor-mentee matching, and real-time communication, ensuring a seamless user experience.

The Edunexus Platform significantly reduces the need for offline interaction by providing a fully digital platform for mentorship. Users can connect, communicate, and share knowledge from any location without the need for physical meetings. This eliminates geographical barriers and saves time and resources for both mentors and mentees.

The platform offers features such as real-time messaging, notifications, and profile-based matching, which enable efficient and continuous interaction online. Users can access guidance, ask questions, and receive feedback instantly through the system. This reduces dependency on traditional face-to-face mentorship methods.

From the analysis, it is evident that the platform performs efficiently with minimal delays and maintains consistency in communication and feedback. The reduction of offline interaction has improved accessibility and flexibility for users, allowing them to connect and collaborate from anywhere. The system also demonstrates strong security measures, ensuring data protection and user privacy.

However, there is scope for further enhancement, such as implementing advanced recommendation algorithms, improving UI/UX design, and increasing scalability for handling larger user bases. Despite these potential improvements, the system proves to be reliable, user-friendly, and effective in facilitating mentorship.

CHAPTER 10

CONCLUSION AND FUTURE WORK

10.1 CONCLUSION

The Edunexus project successfully achieves its objective of developing a web-based platform that facilitates effective interaction between mentors and mentees. The system provides a centralized solution that overcomes the limitations of traditional mentorship methods by enabling users to connect, communicate, and share knowledge in a structured and efficient manner.

The implementation of key modules such as user authentication, profile management, matching, and communication ensures smooth system operation and a user-friendly experience. The platform demonstrates reliable performance, secure data handling, and efficient resource utilization. Testing results confirm that the system functions correctly under various conditions and meets both functional and non-functional requirements.

In addition to these enhancements, the system can be further improved by incorporating features such as automated scheduling and smart reminders to help users manage mentorship sessions efficiently. Integration with cloud-based storage can allow secure handling of user data, session records, and learning materials, ensuring easy access and scalability. The platform can also introduce feedback and rating mechanisms where mentees can evaluate mentors, helping maintain quality and transparency.

Additionally, the project highlights the importance of modern web technologies in building scalable and accessible solutions. While there is scope for further enhancements, such as advanced recommendation systems and improved user interface design, the current system provides a strong foundation for future development.

Overall, the Edunexus Platform serves as an effective and practical solution for mentorship, promoting continuous learning, collaboration, and professional growth in a digital environment.

10.2 FUTURE ENHANCEMENTS

The system can be further enhanced by integrating advanced machine learning algorithms to improve the accuracy of mentor-mentee matching based on user behavior, preferences, and interaction history. This will enable the platform to provide more personalized recommendations, ensuring better guidance and improved user satisfaction. Additionally, incorporating built-in video conferencing features will allow mentors and mentees to communicate in real time without relying on external tools, making sessions more interactive and effective through options like screen sharing and live discussions.

Developing dedicated mobile applications for Android and iOS platforms will significantly enhance accessibility, allowing users to access the system anytime and anywhere with greater convenience. Push notifications and mobile-friendly features will improve engagement and usability. Furthermore, implementing multi-language support will make the platform accessible to a wider audience by enabling users to interact in their preferred language, thereby improving inclusivity and user experience.

To ensure data safety and user trust, enhanced security features such as multi-factor authentication (MFA) and advanced encryption techniques can be implemented. These measures will protect sensitive user information from unauthorized access and potential cyber threats. In addition, an advanced analytics dashboard can be introduced to provide users with detailed insights into their performance and progress through visual reports and data tracking. This will help mentees monitor their learning journey and assist mentors in evaluating their effectiveness, ultimately leading to continuous improvement of the platform.

APPENDIX – A

SOURCE CODE

Auth.py

```

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from datetime import datetime, timedelta
from jose import JWTError, jwt
from pydantic import BaseModel
from fastapi.security import OAuth2PasswordBearer
from database import get_db
from config import settings
import models

router = APIRouter()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login",
auto_error=False)

class LoginRequest(BaseModel):
    email: str
    password: str

# verify password
def verify_password(plain, hashed):
    return plain == hashed

def hash_password(password):
    return password

# create jwt token
def create_token(data: dict):
    exp = datetime.utcnow()
    + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    return jwt.encode(**data, "exp": exp}, settings.SECRET_KEY,
```

```

algorithm=settings.ALGORITHM)

# get logged in user

def get_current_user(token: str = Depends(oauth2_scheme), db: Session =
Depends(get_db)):
    if not token:
        raise HTTPException(status_code=401, detail="Not authenticated")
    try:
        payload = jwt.decode(token, settings.SECRET_KEY,
algorithm=[settings.ALGORITHM])
        user_id = payload.get("sub")
        if not user_id:
            raise HTTPException(status_code=401)
        user = db.query(models.User).filter(models.User.id == int(user_id)).first()
        if not user:
            raise HTTPException(status_code=401)
        return user
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")

# LOGIN

@router.post("/login")
def login(data: LoginRequest, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.email == data.email).first()
    if not user:
        raise HTTPException(status_code=401, detail="Invalid email or password")
    # Plain text comparison
    if data.password != user.password_hash:
        raise HTTPException(status_code=401, detail="Invalid email or password")
    token = create_token({
        "sub": str(user.id),
        "role": user.role

```

```

    })
    return {
        "access_token": token,
        "token_type": "bearer",
        "role": user.role,
        "user_id": user.id,
        "first_login": user.first_login
    }
# GET CURRENT USER INFO
@router.get("/me")
def get_me(current_user: models.User = Depends(get_current_user), db: Session = Depends(get_db)):
    role = current_user.role

    if role == "hod":
        return {"entity_id": current_user.id, "name": "HOD Admin", "role": role}
    elif role == "mentor":
        mentor = db.query(models.Mentor).filter(models.Mentor.user_id ==
current_user.id).first()
        if not mentor:
            raise HTTPException(404, "Mentor record not found")
        staff = db.query(models.Staff).filter(models.Staff.user_id ==
current_user.id).first()
        return {"entity_id": mentor.id, "name": mentor.name, "role": role,
                "staff_id": staff.id if staff else None}
    elif role == "advisor":
        advisor = db.query(models.Advisor).filter(models.Advisor.user_id ==
current_user.id).first()
        if not advisor:
            raise HTTPException(404, "Advisor record not found")
        staff = db.query(models.Staff).filter(models.Staff.user_id ==

```

```

current_user.id).first()

    return {"entity_id": advisor.id, "name": advisor.name, "role": role,
           "year": advisor.year, "section": advisor.section,
           "staff_id": staff.id if staff else None}

elif role == "student":
    student = db.query(models.Student).filter(models.Student.user_id ==
current_user.id).first()

    if not student:
        raise HTTPException(404, "Student record not found")

    return {"entity_id": student.id, "name": student.name, "role": role}

raise HTTPException(400, "Unknown role")

class ChangePasswordReq(BaseModel):
    old_password: str
    new_password: str

@router.post("/change-password")
def change_password(
    req: ChangePasswordReq,
    current_user: models.User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    if not verify_password(req.old_password, current_user.password_hash):
        raise HTTPException(400, "Old password is incorrect")

    if len(req.new_password) < 6:
        raise HTTPException(400, "New password must be at least 6 characters")

    if req.old_password == req.new_password:
        raise HTTPException(400, "New password must be different from old password")

    current_user.password_hash = hash_password(req.new_password)
    db.commit()

    return {"message": "Password changed successfully"}

```

Main.py

```

from fastapi import FastAPI

from fastapi.middleware.cors import CORSMiddleware

from fastapi.staticfiles import StaticFiles

import os

from database import Base, engine

from routers import auth, hod, advisor, mentor, student, chat ,exam , timetable

# Create all tables

Base.metadata.create_all(bind=engine)

# Ensure uploads folder exists

os.makedirs("uploads", exist_ok=True)

app = FastAPI(title="MentorConnect API", version="2.0.0")

# CORS — allow frontend to call the API

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],      # In production: specify your frontend URL
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Serve uploaded certificate files

app.mount("/uploads", StaticFiles(directory="uploads"), name="uploads")

# Register all routers

app.include_router(auth.router, prefix="/api/auth", tags=["Auth"])
app.include_router(hod.router, prefix="/api/hod", tags=["HOD"])
app.include_router(advisor.router, prefix="/api/advisor", tags=["Advisor"])
app.include_router(mentor.router, prefix="/api/mentor", tags=["Mentor"])
app.include_router(student.router, prefix="/api/student", tags=["Student"])
app.include_router(chat.router, prefix="/api/chat", tags=["Chat"])
app.include_router(exam.router, prefix="/api/exam", tags=["Exam"])

```

```

app.include_router(timetable.router, prefix="/api/timetable", tags=["Timetable"])

@app.get("/")
def root():

    return {"message": "MentorConnect API is running. Visit /docs for API
documentation."}

```

Index.html

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>MentorConnect — Login</title>

    <link rel="stylesheet" href="assets/style.css">

</head>

<body class="login-body">

    <div class="login-screen">

        <div class="login-card">

            <div class="login-logo">

                <div class="logo-icon">M</div>

                <div>

                    <div class="login-title">MentorConnect</div>

                    <div class="login-sub">College Mentor–Mentee Management System</div>

                </div>

            </div>

            <div class="login-divider"></div>

            <div class="form-group">

                <label class="form-label">Email Address</label>

                <input type="email" id="loginEmail" class="form-input"

placeholder="your@college.edu">

            </div>

            <div class="form-group">

```



```

    <label class="form-label">Password</label>

    <input type="password" id="loginPwd" class="form-input"
placeholder="Enter password">

    <div id="loginError" class="error-msg" style="display:none"></div>

</div>

<button class="btn-login" id="loginBtn" onclick="doLogin()">
    <span id="loginBtnText">Sign In</span>
    <span id="loginSpinner" style="display:none">Signing in...</span>
</button>

</div>
</div>

<script src="assets/api.js"></script>
<script>
    // If already logged in, go straight to dashboard
    if (localStorage.getItem('token')) window.location.href = 'dashboard.html';

    async function doLogin() {
        const email = document.getElementById('loginEmail').value.trim();
        const pwd = document.getElementById('loginPwd').value;
        const errEl = document.getElementById('loginError');
        const btn = document.getElementById('loginBtn');

        if (!email || !pwd) {
            errEl.textContent = 'Please enter email and password.';
            errEl.style.display = 'block';
            return;
        }
    }

```

```

document.getElementById('loginBtnText').style.display = 'none';
document.getElementById('loginSpinner').style.display = 'inline';
btn.disabled = true;
errEl.style.display = 'none';

// Allow pressing Enter to submit
document.addEventListener('keydown', function(e) {
  if (e.key === 'Enter') doLogin();
});
</script>
</body>
</html>

```

Config.py

```

from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    DATABASE_URL: str
    SECRET_KEY: str
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 480
    UPLOAD_DIR: str = "./uploads"

    class Config:
        env_file = ".env"

settings = Settings()

```

APPENDIX – B

SCREENSHOTS

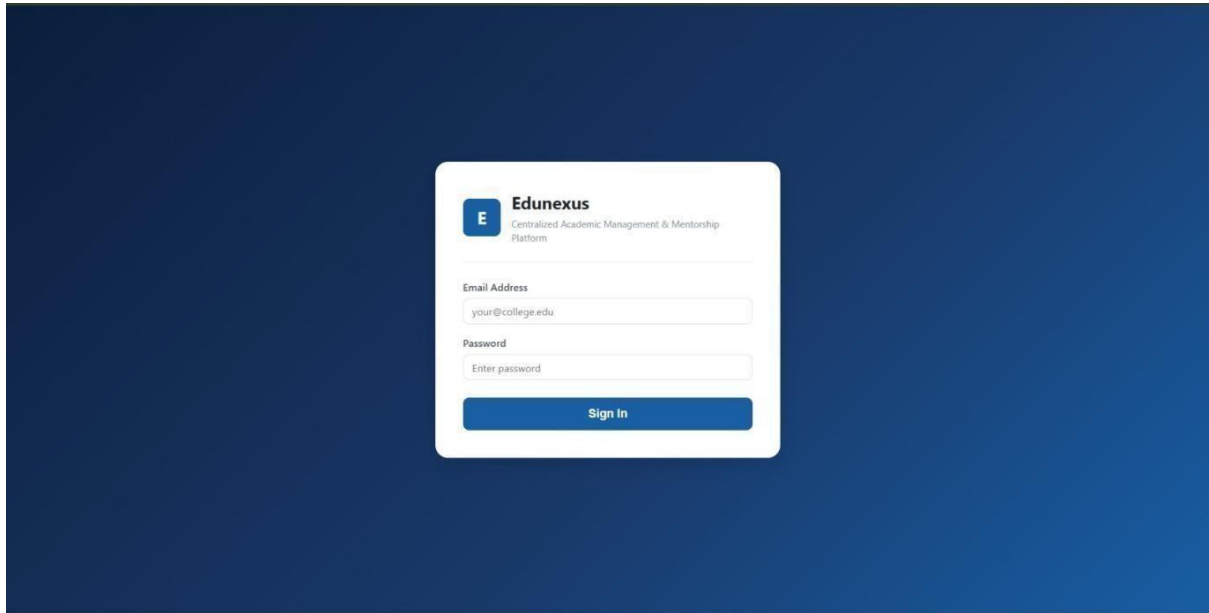


Figure B.1 Login page

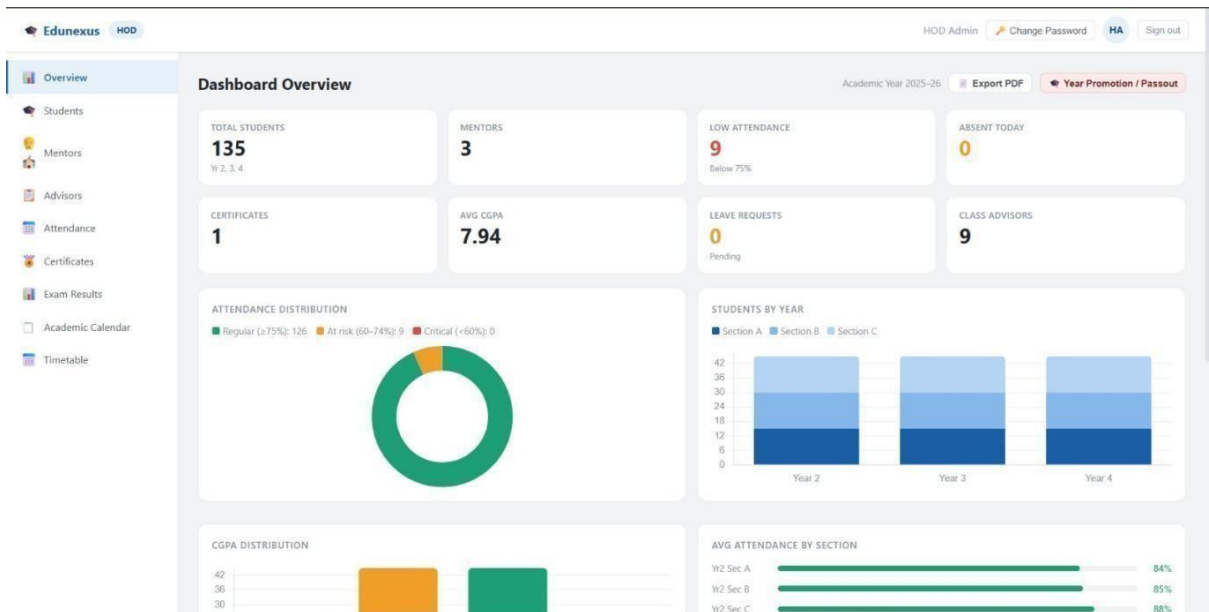


Figure B.2 HOD Dashboard page

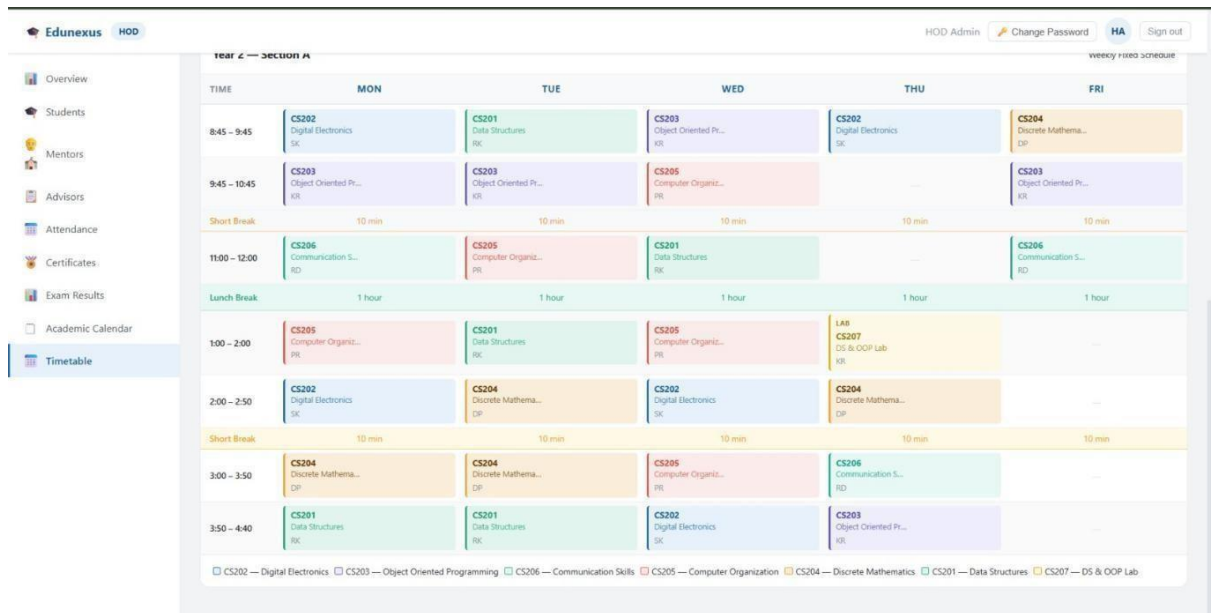


Figure B.3 Time Table generation

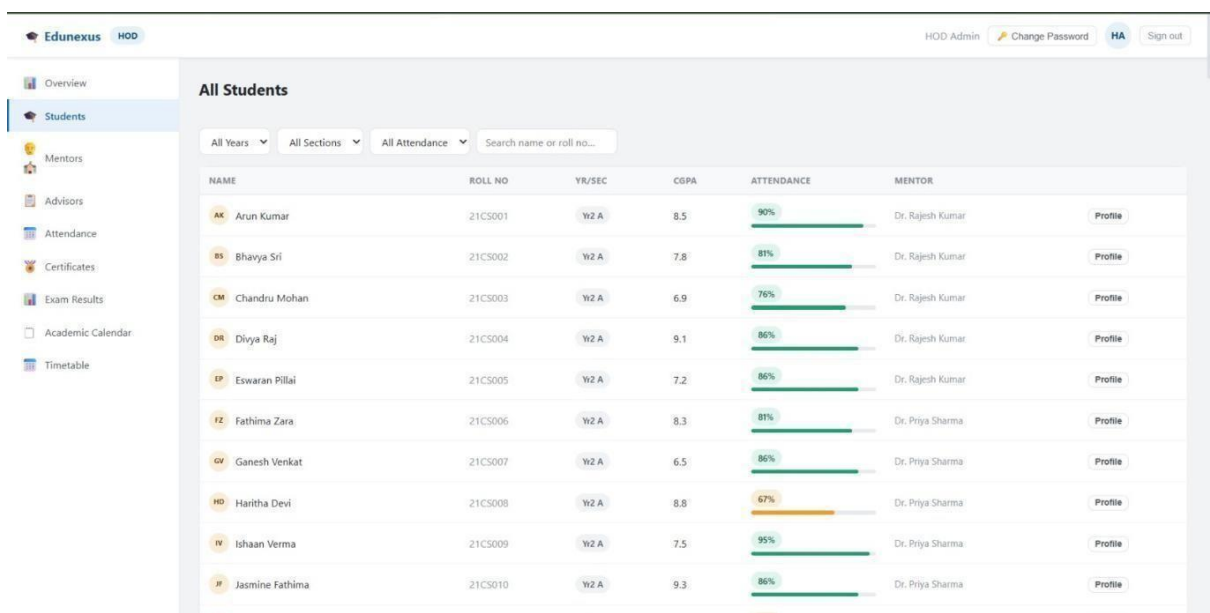


Figure. B.4 Students overview page

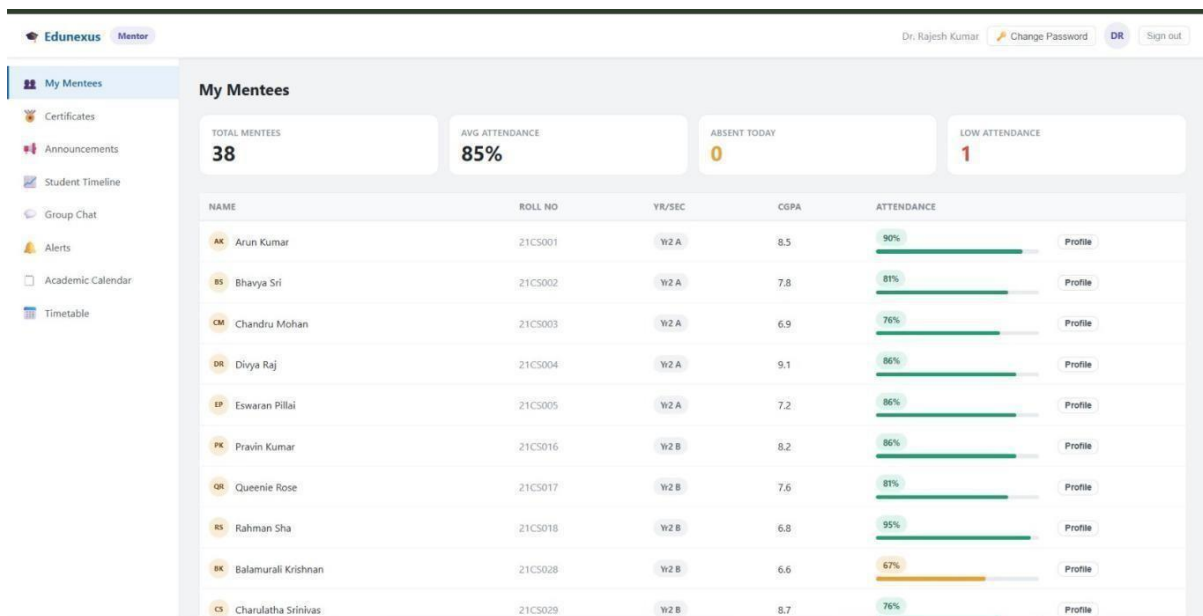


Figure B.5 Mentor Dashboard

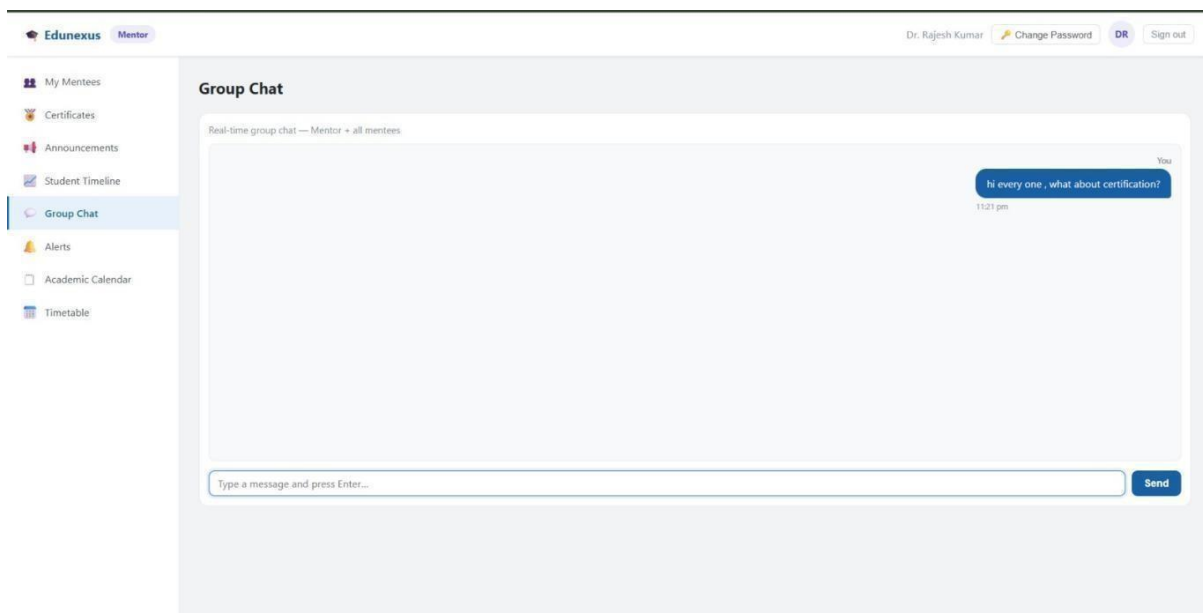


Figure B.6 Chat box

Edunexus **Class Advisor** Mrs. Kavitha Rajan Change Password MX Sign out

Tuesday — 24 Mar 2026 Your periods for that day load automatically

Tuesday — Your 4 period(s)

9:45 – 10:45
Object Oriented Programming | Year 2 — Section A Not marked yet

✓ All Present X All Absent

NAME	ROLL NO	STATUS
Arun Kumar	21CS001	Present
Bhavya Sri	21CS002	Present
Chandru Mohan	21CS003	Present
Divya Raj	21CS004	Present
Eswaran Pillai	21CS005	Present
Fathima Zara	21CS006	Present
Ganesh Venkat	21CS007	Present
Haritha Devi	21CS008	Present
Ishaan Verma	21CS009	Present
Jasmine Fathima	21CS010	Present
Kiran Sabu	21CS011	Present

Figure B.7 Period-wise attendance

Edunexus **Student** Pravin Kumar Change Password PK Sign out

My Profile

PK Pravin Kumar
21CS016 - Year 2 - Section B

EMAIL pravin.k@student.edu	PHONE 9876543216
DATE OF BIRTH 11 Mar 2003	CGPA 8.2
BACKLOGS 0	ATTENDANCE 86%
MENTOR Dr. Rajesh Kumar	ADVISOR Mr. Suresh Babu

Achievements
No achievements added

Exam Marks
No exam marks available yet.

Certificates

Figure B.7 Students Dashboard

REFERENCES

- [1] Fielding, R. T., “Architectural Styles and the Design of Network-Based Software Architectures,” Doctoral Dissertation, University of California, Irvine, pp. 1–162, 2000
- [2] Tilkov, S., & Vinoski, S., “Node.js: Using JavaScript to Build High-Performance Network Programs,” *IEEE Internet Computing*, vol. 14, no. 6, pp. 80–83, 2010
- [3] Pautasso, C., Zimmermann, O., & Leymann, F., “RESTful Web Services vs. Big Web Services: Making the Right Architectural Decision,” *Proceedings of the 17th International World Wide Web Conference*, pp. 805–814, 2008
- [4] Hardt, D., “The OAuth 2.0 Authorization Framework,” *IETF RFC 6749*, pp. 1–76, 2012
- [5] Rescorla, E., “HTTP Over TLS (HTTPS),” *IETF RFC 2818*, pp. 1–15, 2000
- [6] Banks, A., & Gupta, R., “MQTT Version 3.1.1,” *OASIS Standard*, pp. 1–81, 2014
- [7] Gamma, E., Helm, R., Johnson, R., & Vlissides, J., “Design Patterns: Elements of Reusable Object-Oriented Software,” Addison-Wesley, pp. 1–395, 1994
- [8] Freeman, E., Robson, E., Sierra, K., & Bates, B., “Head First Design Patterns,” O’Reilly Media, pp. 1–694, 2004
- [9] Kruchten, P., “The 4+1 View Model of Architecture,” *IEEE Software*, vol. 12, no. 6, pp. 42–50, 1995
- [10] Bass, L., Clements, P., & Kazman, R., “Software Architecture in Practice,” Addison-Wesley, pp. 1–560, 2012
- [11] Sommerville, I., “Software Engineering,” Pearson Education, 10th ed., pp. 1–816, 2015
- [12] Pressman, R. S., & Maxim, B. R., “Software Engineering: A Practitioner’s Approach,” McGraw-Hill Education, 8th ed., pp. 1–940, 2014

- [13] Dean, J., & Ghemawat, S., “MapReduce: Simplified Data Processing on Large Clusters,” *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008
- [14] Hohpe, G., & Woolf, B., “Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions,” Addison-Wesley, pp. 1–735, 2003
- [15] Fowler, M., “Patterns of Enterprise Application Architecture,” Addison-Wesley, pp. 1–533, 2002